# UTS

# PEMBELAJARAN MESIN




NAMA : Muh. Razzan Rezqy Pratama

NIM : 202431185

KELAS : G

DOSEN : Dr. Dra. Dwina Kuswardani, M.Kom

NO.PC :

ASISTEN : 1. Muhammad Ichwan Fauzi

2. Morgan Immanuel Nainggolan

## INSTITUT TEKNOLOGI PLN

## TEKNIK INFORMATIKA

## 2026/2027

## DAFTAR ISI

**BAB I PENDAHULUAN ..... 3**
1.1 Rumusan Masalah ..... 3
1.2 Tujuan Masalah ..... 3
1.3 Manfaat Masalah ..... 4
**BAB II LANDASAN TEORI ..... 5**
2.1 Pembelajaran Terawasi: Klasifikasi dan Regresi ..... 5
2.2 Dataset Bank Marketing pada Penelitian Terdahulu ..... 5
2.3 Exploratory Data Analysis dan Preprocessing ..... 5
2.4 Decision Tree ..... 6
2.5 Regresi Linear ..... 7
2.6 Metrik Evaluasi ..... 7
**BAB III HASIL ..... 9**
3.1 Persiapan Library dan Watermark NIM ..... 9
3.2 Membaca Dataset ..... 9
3.3 Exploratory Data Analysis (EDA) ..... 9
3.4 Preprocessing Data ..... 16
3.5 Decision Tree (Klasifikasi) ..... 17
3.6 Regresi Linear ..... 22
3.7 Rangkuman Hasil ..... 26
3.8 Pembahasan ..... 27
**BAB IV PENUTUP ..... 29**
4.1 Kesimpulan ..... 29
4.2 Saran ..... 29
**DAFTAR PUSTAKA ..... 31**

# BAB I
# PENDAHULUAN

## 1.1 Rumusan Masalah

Deposito berjangka merupakan salah satu produk perbankan yang dipasarkan bank melalui kampanye pemasaran langsung, misalnya melalui telepon. Karena tidak semua nasabah yang dihubungi bersedia berlangganan, bank perlu mengenali karakteristik nasabah yang berpeluang berlangganan agar kampanye lebih efisien. Penelitian terdahulu telah memanfaatkan algoritma pembelajaran mesin untuk memprediksi langganan deposito bank (Hou et al., 2022; Hayder et al., 2023). Pada UTS ini digunakan dataset Bank Marketing (04_bank_customers.csv) yang berisi 45.211 nasabah dengan 17 atribut. Dua tugas dikerjakan, yaitu klasifikasi menggunakan algoritma Decision Tree dan regresi menggunakan algoritma Regresi Linear.

Berdasarkan latar belakang tersebut, rumusan masalah pada UTS ini adalah sebagai berikut.

1. Bagaimana karakteristik data nasabah bank berdasarkan hasil EDA, meliputi distribusi variabel, data kosong, outlier, dan korelasi antar variabel?
2. Bagaimana tahapan preprocessing yang diperlukan agar data dapat dilatih menggunakan algoritma Decision Tree dan Regresi Linear?
3. Seberapa akurat algoritma Decision Tree dalam mengklasifikasikan nasabah yang berlangganan deposito berjangka (variabel y)?
4. Seberapa baik algoritma Regresi Linear dalam memprediksi saldo nasabah (variabel balance)?
5. Variabel apa yang paling berpengaruh terhadap hasil prediksi pada kedua model?

## 1.2 Tujuan Masalah

Tujuan dan hasil yang diperoleh dari pengerjaan UTS ini adalah sebagai berikut.

1. Melakukan EDA dan menemukan karakteristik data: tidak ada NaN maupun data duplikat, tetapi terdapat nilai unknown yang besar pada poutcome (81,75%) dan contact (28,80%), kelas target tidak seimbang (88,30% no dan 11,70% yes), serta outlier pada beberapa variabel numerik.
2. Menyiapkan data melalui pembagian data 80:20 secara stratifikasi, One-Hot Encoding, dan Pipeline sehingga proses preprocessing hanya belajar dari data latih.

3. Membangun model Decision Tree dan memperoleh akurasi 89,86% pada data uji dengan ROC-AUC 0,863 setelah tuning hyperparameter (gini, max_depth = 7). Model awal tanpa batasan mengalami overfitting (akurasi latih 100% dan uji 87,37%).
4. Membangun model Regresi Linear untuk memprediksi balance dan memperoleh R-squared 0,0468 dengan MAE 1.486,86 dan RMSE 3.055,92 euro pada data uji.
5. Mengetahui fitur paling berpengaruh pada Decision Tree, yaitu duration (0,485) dan poutcome_success (0,254), serta menjelaskan setiap cell kode beserta outputnya.

1.3 Manfaat Masalah

Manfaat yang diperoleh dari hasil UTS ini adalah sebagai berikut.

1. Bagi mahasiswa, UTS ini melatih alur kerja pembelajaran mesin secara utuh, mulai dari EDA, preprocessing, pelatihan model, hingga evaluasi dan interpretasi hasil.
2. Bagi pihak perbankan, hasil analisis memberi gambaran segmen nasabah yang lebih responsif terhadap penawaran deposito, misalnya pelajar, pensiunan, dan nasabah dengan hasil kampanye sebelumnya yang berhasil.
3. Bagi pengembangan keilmuan, hasil ini menunjukkan pentingnya memilih metrik yang tepat pada data tidak seimbang, mewaspadai fitur yang baru diketahui setelah kejadian (*data leakage*), dan mengenali batas kemampuan Regresi Linear pada hubungan yang lemah.

# BAB II
# LANDASAN TEORI

## 2.1 Pembelajaran Terawasi: Klasifikasi dan Regresi

Pembelajaran terawasi (*supervised learning*) adalah pendekatan pembelajaran mesin yang melatih model dengan data berlabel, yaitu pasangan fitur (variabel bebas) dan target (variabel terikat) yang sudah diketahui, sehingga model dapat memprediksi target pada data baru. Abu-Faraj et al. (2022) menempatkan regresi sebagai bagian dari pembelajaran terawasi yang bertujuan menemukan hubungan antara variabel bebas dan variabel terikat (respons).

Berdasarkan jenis target, tugas pembelajaran terawasi dibagi menjadi klasifikasi, yaitu memprediksi kategori diskret seperti nasabah berlangganan atau tidak, dan regresi, yaitu memprediksi nilai kontinu seperti saldo nasabah. Pada UTS ini, Decision Tree digunakan untuk klasifikasi dan Regresi Linear digunakan untuk regresi.

## 2.2 Dataset Bank Marketing pada Penelitian Terdahulu

Dataset Bank Marketing berasal dari UCI Machine Learning Repository dan berkaitan dengan kampanye pemasaran langsung sebuah bank di Portugal; tugas klasifikasinya adalah memprediksi apakah nasabah akan berlangganan deposito berjangka (Hayder et al., 2023). Hayder et al. (2023) mencatat bahwa dataset ini memuat nilai unknown dan kelasnya tidak seimbang.

Hou et al. (2022) memprediksi langganan deposito bank menggunakan lima algoritma terawasi, yaitu Naive Bayes, Decision Tree, Random Forest, Support Vector Machine, dan Neural Network, lalu membandingkan kinerjanya. Hayder et al. (2023) membandingkan k-NN, Decision Tree, Naive Bayes, dan SVM, dengan Decision Tree memberikan hasil terbaik (akurasi 91%) dan SVM memperoleh akurasi 89%. Perlu dicatat bahwa penelitian tersebut memakai versi dataset yang berbeda dari yang digunakan pada UTS ini, sehingga angkanya tidak dibandingkan secara langsung.

## 2.3 Exploratory Data Analysis dan Preprocessing

*Exploratory Data Analysis* (EDA) adalah tahap memahami data sebelum pemodelan, antara lain melalui statistik deskriptif, histogram, boxplot, dan matriks korelasi. Pada tahap ini diperiksa tipe data, data kosong, duplikat, outlier, dan keseimbangan kelas. Salah satu cara mendeteksi outlier adalah aturan IQR, yaitu nilai di bawah Q1 - 1,5 x IQR atau di atas Q3 + 1,5 x IQR dianggap outlier.

Preprocessing mengubah data mentah agar dapat diproses algoritma. Sebagian besar algoritma pembelajaran mesin dirancang untuk masukan numerik sehingga variabel kategorikal perlu dikodekan. Pargent et al. (2022) menjelaskan bahwa pengkodean indikator (*one-hot* atau *dummy*) memetakan setiap level kategori menjadi variabel biner yang menandai ada atau tidaknya level tersebut; kekurangannya adalah jumlah fitur bertambah satu untuk setiap level. Abu-Faraj et al. (2022) juga menyebutkan penyaringan dan normalisasi data sebagai langkah yang dilakukan bila diperlukan sebelum pemodelan.

Pembagian data menjadi data latih dan data uji memungkinkan model dinilai pada data yang tidak dipakai untuk belajar. Pada data tidak seimbang, pembagian berstrata (*stratified*) menjaga proporsi kelas tetap sama pada kedua bagian. Pemilihan hyperparameter dilakukan dengan validasi silang (*cross-validation*), yang menurut Halabaku dan Bytyçi (2024) membutuhkan waktu komputasi yang cukup besar.

## 2.4 Decision Tree

Decision Tree adalah algoritma yang membangun struktur pohon untuk mengambil keputusan dengan merangkum sekumpulan aturan klasifikasi dari data. Node akar dan node internal menerapkan aturan pemisah pada suatu fitur, sedangkan node daun menyatakan kelas hasil prediksi, biasanya berdasarkan suara terbanyak pada daun tersebut (Halabaku & Bytyçi, 2024). Decision Tree dapat menangani atribut nominal, ordinal, maupun kontinu, dan aturannya mudah ditafsirkan (Alsulami et al., 2026).

Pemilihan pemisah pada setiap node didasarkan pada ukuran impuritas. Alsulami et al. (2026) menguraikan tiga ukuran umum: Gini impurity, entropi, dan kesalahan klasifikasi. Gini bernilai 0 bila seluruh data pada node berasal dari satu kelas, dan pada kasus biner mencapai maksimum 0,5 bila kedua kelas sama banyak.

$$Gini(t) = 1 - \Sigma\, p_j^2$$

$$Entropi(S) = - \Sigma\, p(x)\, \log_2 p(x)$$

$$Kesalahan\ klasifikasi = 1 - maks(p_i)$$

Pada rumus tersebut $p_j$ adalah proporsi data kelas j pada node. Pemisah terbaik dipilih sebagai yang paling menurunkan impuritas.

Kelemahan utama Decision Tree adalah kecenderungan *overfitting*, terutama bila pohon dibiarkan tumbuh tanpa batas sehingga membentuk aturan yang sangat spesifik pada data latih dan buruk pada data baru (Halabaku & Bytyçi, 2024; Alsulami et al., 2026). Pemangkasan (*pruning*) dan pembatasan kompleksitas pohon, seperti max_depth dan min_samples_leaf, mengurangi kedalaman dan jumlah node sehingga kemampuan

generalisasi meningkat meskipun akurasi data latih dapat sedikit menurun (Halabaku & Bytyçi, 2024). Tingkat kepentingan fitur dapat dihitung sebagai jumlah perolehan informasi yang diberikan suatu fitur setiap kali menjadi pemisah (Halabaku & Bytyçi, 2024). Selain itu, kinerja Decision Tree dapat menurun pada data yang tidak seimbang atau mengandung derau (Alsulami et al., 2026), dan model dapat cenderung bias ke kelas mayoritas (Halabaku & Bytyçi, 2024).

## 2.5 Regresi Linear

Regresi Linear memodelkan hubungan antara satu atau lebih variabel bebas dengan variabel terikat (respons) berupa nilai kontinu (Abu-Faraj et al., 2022). Pada regresi linear berganda, prediksi dinyatakan sebagai kombinasi linear fitur:

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \ldots + \beta_p x_p$$

dengan $\beta_0$ sebagai intercept dan $\beta_j$ sebagai koefisien fitur ke-j. Koefisien diestimasi dengan meminimalkan galat kuadrat antara nilai prediksi dan nilai asli; kualitas model diukur antara lain dengan *Mean Square Error* (MSE) (Abu-Faraj et al., 2022). Koefisien $\beta_j$ menyatakan perubahan prediksi ketika fitur ke-j naik satu satuan dengan fitur lain tetap.

Keterbatasan Regresi Linear adalah asumsi hubungan yang linear. Abu-Faraj et al. (2022) mengingatkan bahwa hubungan antara target dan variabel prediktor pada banyak kasus bersifat tidak linear sehingga diperlukan teknik yang lebih canggih.

## 2.6 Metrik Evaluasi

Evaluasi klasifikasi bertumpu pada *confusion matrix* yang berisi *True Positive* (TP), *True Negative* (TN), *False Positive* (FP), dan *False Negative* (FN). Dari sini dihitung accuracy, precision, recall, dan F1-score. ROC-AUC mengukur kemampuan model membedakan kedua kelas pada berbagai ambang probabilitas. Hancock et al. (2023) menunjukkan bahwa pemilihan metrik yang kurang tepat pada data sangat tidak seimbang dapat menyembunyikan informasi penting; misalnya AUC tidak menangkap informasi tentang precision dan jumlah *false positive* yang diungkap oleh AUPRC.

$$Accuracy = (TP + TN) / (TP + TN + FP + FN)$$

$$Precision = TP / (TP + FP)$$

$$Recall = TP / (TP + FN)$$

$$F1 = 2 \times Precision \times Recall / (Precision + Recall)$$

Evaluasi regresi memakai MAE, RMSE, dan R-squared. MAE adalah rata-rata selisih absolut, RMSE adalah akar dari rata-rata kuadrat selisih sehingga lebih sensitif terhadap kesalahan besar, dan R-squared menyatakan seberapa besar variasi variabel

terikat yang dapat dijelaskan model; nilai yang lebih tinggi menunjukkan model yang lebih baik (Rosita & Moonlight, 2024). Pemilihan antara RMSE dan MAE bergantung pada distribusi galat (Karunasingha, 2022), dan karena masih terdapat perbedaan pandangan tentang penggunaannya, praktik yang lazim adalah melaporkan keduanya (Hodson, 2022).

$$MAE = (1/n) \Sigma |y_i - \hat{y}_i|$$

$$RMSE = \sqrt{[(1/n) \Sigma (y_i - \hat{y}_i)^2]}$$

$$R^2 = 1 - \Sigma (y_i - \hat{y}_i)^2 / \Sigma (y_i - \bar{y})^2$$

# BAB III
# HASIL

Bab ini menyajikan kode program beserta output dan penjelasannya. Pengerjaan memakai bahasa pemrograman Python pada Jupyter Notebook dengan library pandas, numpy, matplotlib, seaborn, dan scikit-learn. Seluruh cell kode diberi watermark NIM dalam tiga bentuk: komentar pada baris pertama cell, baris [ NIM | Nama ] pada output teks, dan tulisan transparan pada setiap grafik. Cell diberi nomor sesuai urutan kode pada file notebook 202431185_UTS_PM.ipynb.

## 3.1 Persiapan Library

[58]:
```
# NIM: 202431185 | Muh. Razzan Rezqy Pratama
import warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.linear_model import LinearRegression
from sklearn.metrics import (accuracy_score, precision_score, recall_score, f1_score, classification_report,
                             confusion_matrix, roc_auc_score, roc_curve,
                             mean_absolute_error, mean_squared_error, r2_score)
```

Cell ini mengimpor seluruh library yang dibutuhkan. pandas dan numpy dipakai untuk mengolah data, matplotlib dan seaborn untuk visualisasi, dan modul sklearn untuk pembagian data (train_test_split, GridSearchCV), preprocessing (OneHotEncoder, StandardScaler, ColumnTransformer, Pipeline), algoritma (DecisionTreeClassifier, LinearRegression), serta metrik evaluasi.

## 3.2 Membaca Dataset dan Exploratory Data Analysis (EDA)

*Kode program:*

[59]:
```
# NIM: 202431185 | Muh. Razzan Rezqy Pratama
df = pd.read_csv("04_bank_customers.csv", sep=";")
```

[61]:
```
print("Jumlah baris & kolom :", df.shape)
```

```
Jumlah baris & kolom : (45211, 17)
```

[62]:
```
df.head()
```

[62]:

| | age | job | marital | education | default | balance | housing | loan | contact | day | month | duration | campaign | pdays | previous | poutcome | y |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 58 | management | married | tertiary | no | 2143 | yes | no | unknown | 5 | may | 261 | 1 | -1 | 0 | unknown | no |
| 1 | 44 | technician | single | secondary | no | 29 | yes | no | unknown | 5 | may | 151 | 1 | -1 | 0 | unknown | no |
| 2 | 33 | entrepreneur | married | secondary | no | 2 | yes | yes | unknown | 5 | may | 76 | 1 | -1 | 0 | unknown | no |
| 3 | 47 | blue-collar | married | unknown | no | 1506 | yes | no | unknown | 5 | may | 92 | 1 | -1 | 0 | unknown | no |
| 4 | 33 | unknown | single | unknown | no | 1 | no | no | unknown | 5 | may | 198 | 1 | -1 | 0 | unknown | no |

pd.read_csv("04_bank_customers.csv", sep=";") membaca file CSV ke dalam DataFrame df. Parameter sep=";" wajib ditulis karena file ini memakai tanda titik koma sebagai pemisah kolom; tanpa parameter tersebut seluruh isi baris akan terbaca sebagai satu kolom. df.shape menampilkan jumlah baris dan kolom, sedangkan df.head() menampilkan lima baris pertama untuk melihat gambaran isi data. Dataset terdiri dari 45.211 baris dan 17 kolom (16 fitur + 1 target y). Setiap baris adalah satu nasabah bank yang dihubungi dalam kampanye pemasaran deposito berjangka.

```
[5]: # NIM: 202431185 | Muh. Razzan Rezqy Pratama
     df.info()

<class 'pandas.DataFrame'>
RangeIndex: 45211 entries, 0 to 45210
Data columns (total 17 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   age        45211 non-null  int64
 1   job        45211 non-null  str
 2   marital    45211 non-null  str
 3   education  45211 non-null  str
 4   default    45211 non-null  str
 5   balance    45211 non-null  int64
 6   housing    45211 non-null  str
 7   loan       45211 non-null  str
 8   contact    45211 non-null  str
 9   day        45211 non-null  int64
 10  month      45211 non-null  str
 11  duration   45211 non-null  int64
 12  campaign   45211 non-null  int64
 13  pdays      45211 non-null  int64
 14  previous   45211 non-null  int64
 15  poutcome   45211 non-null  str
 16  y          45211 non-null  str
dtypes: int64(7), str(10)
memory usage: 8.1 MB
```

df.info() menampilkan ringkasan struktur data: jumlah baris, nama setiap kolom, jumlah nilai non-null, dan tipe data. Informasi ini dipakai untuk membedakan kolom numerik (int64) dan kolom kategorikal (teks), serta untuk melakukan pengecekan awal terhadap data kosong.

7 kolom bertipe numerik (int64) dan 10 kolom bertipe teks/kategorikal. Seluruh kolom memiliki 45.211 nilai non-null, jadi tidak ada sel kosong (NaN).

```
[9]: # NIM: 202431185 | Muh. Razzan Rezqy Pratama
     df.describe().T
```

[9]:

| | count | mean | std | min | 25% | 50% | 75% | max |
|---|---|---|---|---|---|---|---|---|
| **age** | 45211.0 | 40.936210 | 10.618762 | 18.0 | 33.0 | 39.0 | 48.0 | 95.0 |
| **balance** | 45211.0 | 1362.272058 | 3044.765829 | -8019.0 | 72.0 | 448.0 | 1428.0 | 102127.0 |
| **day** | 45211.0 | 15.806419 | 8.322476 | 1.0 | 8.0 | 16.0 | 21.0 | 31.0 |
| **duration** | 45211.0 | 258.163080 | 257.527812 | 0.0 | 103.0 | 180.0 | 319.0 | 4918.0 |
| **campaign** | 45211.0 | 2.763841 | 3.098021 | 1.0 | 1.0 | 2.0 | 3.0 | 63.0 |
| **pdays** | 45211.0 | 40.197828 | 100.128746 | -1.0 | -1.0 | -1.0 | -1.0 | 871.0 |
| **previous** | 45211.0 | 0.580323 | 2.303441 | 0.0 | 0.0 | 0.0 | 0.0 | 275.0 |

df.describe() menghitung jumlah data, rata-rata, standar deviasi, nilai minimum, kuartil (25%, 50%, 75%), dan nilai maksimum untuk setiap kolom numerik. Tambahan .T mentranspose tabel sehingga setiap variabel tampil sebagai satu baris dan tabel lebih mudah dibaca. balance sangat tidak simetris: rata-rata 1.362 jauh di atas median 448, dengan nilai maksimum 102.127. pdays memiliki median -1 (artinya nasabah belum pernah dihubungi pada kampanye sebelumnya), dan duration memiliki rata-rata 258 detik dengan nilai maksimum 4.918 detik.

```
[63]: # NIM: 202431185 | Muh. Razzan Rezqy Pratama

      df.describe(exclude="number").T
```

[63]:

| | count | unique | top | freq |
|---|---|---|---|---|
| **job** | 45211 | 12 | blue-collar | 9732 |
| **marital** | 45211 | 3 | married | 27214 |
| **education** | 45211 | 4 | secondary | 23202 |
| **default** | 45211 | 2 | no | 44396 |
| **housing** | 45211 | 2 | yes | 25130 |
| **loan** | 45211 | 2 | no | 37967 |
| **contact** | 45211 | 3 | cellular | 29285 |
| **month** | 45211 | 12 | may | 13766 |
| **poutcome** | 45211 | 4 | unknown | 36959 |
| **y** | 45211 | 2 | no | 39922 |

Argumen exclude="number" membuat describe() hanya memproses kolom non-numerik. Hasilnya berupa jumlah data (count), jumlah kategori unik (unique), kategori yang paling sering muncul (top), dan frekuensinya (freq). Kategori terbanyak pada job adalah *blue-collar* (9.732), marital adalah *married* (27.214), dan education adalah *secondary* (23.202). Kolom poutcome didominasi nilai unknown (36.959 baris).

```
[12]: # NIM: 202431185 | Muh. Razzan Rezqy Pratama
      df.isnull().sum()
```

[12]:

```
age          0
job          0
marital      0
education    0
default      0
balance      0
housing      0
loan         0
contact      0
day          0
month        0
duration     0
campaign     0
pdays        0
previous     0
poutcome     0
y            0
dtype: int64
```

```
[14]: df.duplicated().sum()
```

[14]:

```
np.int64(0)
```

df.isnull().sum()digunakan untuk menghitung jumlah total nilai yang kosong atau *missing values* pada setiap kolom di dalam DataFrame df. Hasil dari eksekusi kode tersebut menunjukkan angka 0 untuk seluruh kolom, mulai dari kolom age, job, marital, hingga kolom y, yang berarti dataset tersebut bersih dari nilai yang hilang. Selanjutnya, baris kode df.duplicated().sum() dijalankan untuk mendeteksi dan menjumlahkan total baris data yang duplikat di dalam dataset. Hasil dari pengecekan duplikat ini mengembalikan nilai np.int64(0), yang menyimpulkan bahwa tidak ada satupun baris data yang mengalami duplikasi. Secara keseluruhan, kedua pengujian ini menunjukkan bahwa dataset yang dianalisis berada dalam kondisi yang bersih, lengkap, dan siap untuk tahap pengolahan selanjutnya.

[17]:
```
# NIM: 202431185 | Muh. Razzan Rezqy Pratama

kolom_kategori_semua = df.select_dtypes(exclude="number").columns.drop("y")
unknown = pd.DataFrame({
    "jumlah_unknown": [(df[c] == "unknown").sum() for c in kolom_kategori_semua]
}, index=kolom_kategori_semua)
unknown["persen"] = (unknown["jumlah_unknown"] / len(df) * 100).round(2)
unknown
```

[17]:

| | jumlah_unknown | persen |
|---|---|---|
| **job** | 288 | 0.64 |
| **marital** | 0 | 0.00 |
| **education** | 1857 | 4.11 |
| **default** | 0 | 0.00 |
| **housing** | 0 | 0.00 |
| **loan** | 0 | 0.00 |
| **contact** | 13020 | 28.80 |
| **month** | 0 | 0.00 |
| **poutcome** | 36959 | 81.75 |

Berdasarkan dattidak ada NaN dan tidak ada baris duplikat. Namun ada data kosong tersembunyi berupa unknown: poutcome 81,75%, contact 28,80%, education 4,11%, dan job 0,64%. Karena persentase poutcome dan contact sangat besar, baris tidak dihapus dan unknown diperlakukan sebagai kategori tersendiri.

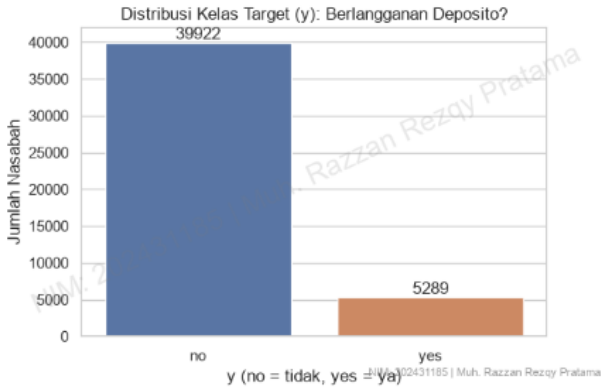
[18]:
```
# NIM: 202431185 | Muh. Razzan Rezqy Pratama

print(df["y"].value_counts())
print()
print((df["y"].value_counts(normalize=True) * 100).round(2).astype(str) + " %")

plt.figure(figsize=(6, 4))
ax = sns.countplot(x="y", hue="y", data=df, legend=False)
for container in ax.containers:
    ax.bar_label(container)            # menampilkan angka di atas batang
plt.title("Distribusi Kelas Target (y): Berlangganan Deposito?")
plt.xlabel("y (no = tidak, yes = ya)")
plt.ylabel("Jumlah Nasabah")
tampil()
```

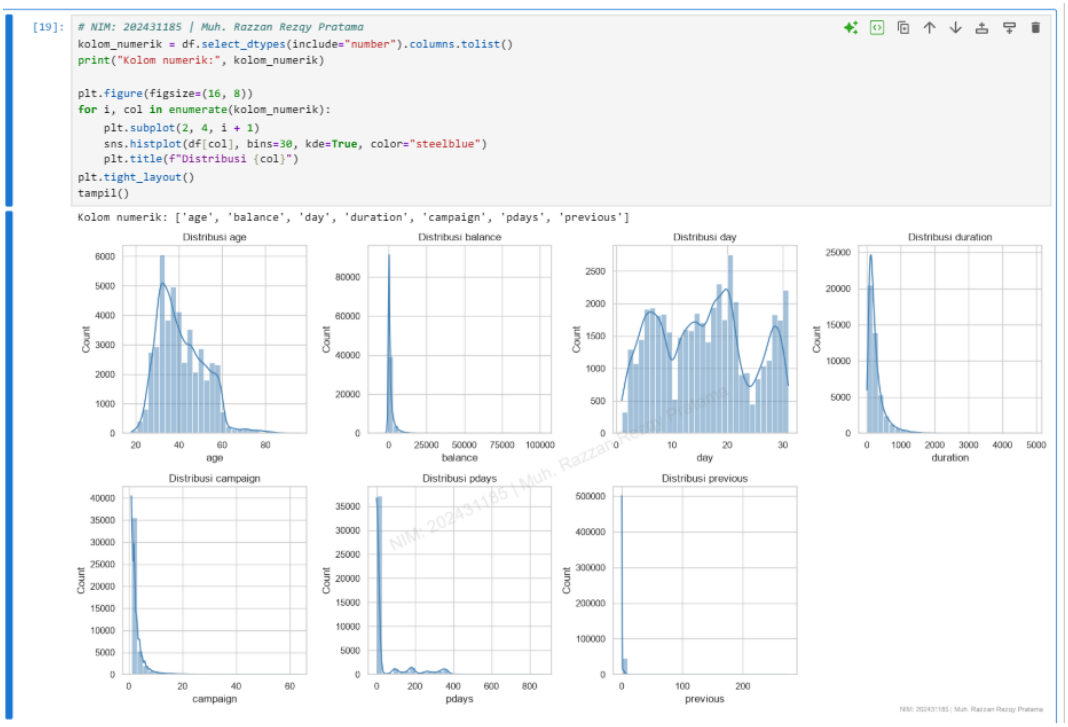
```
y
no     39922
yes     5289
Name: count, dtype: int64

y
no     88.3 %
yes    11.7 %
Name: proportion, dtype: str
```




value_counts() menghitung jumlah nasabah pada tiap kelas, dan normalize=True mengubahnya menjadi proporsi. sns.countplot menggambar diagram batang untuk kelas no dan yes; ax.bar_label menuliskan angka di atas setiap batang; dan tampil() menambahkan

watermark sebelum grafik ditampilkan. Kelas target tidak seimbang: no sebanyak 39.922 (88,30%) dan yes sebanyak 5.289 (11,70%). Artinya, model yang selalu menebak no sudah mendapat akurasi 88,30%, sehingga akurasi saja tidak cukup untuk menilai model. Precision, recall, F1, dan ROC-AUC juga perlu dilihat.




Daftar kolom numerik diambil otomatis dengan select_dtypes(include="number"). Perulangan for menggambar satu histogram untuk setiap variabel pada kisi 2 x 4 (plt.subplot(2, 4, i + 1)). bins=30 membagi rentang nilai menjadi 30 batang dan kde=True menambahkan kurva perkiraan kepadatan agar bentuk distribusi terlihat. balance, duration, campaign, pdays, dan previous condong ke kanan (*right-skewed*), yaitu sebagian besar nilai kecil dengan ekor panjang ke nilai besar. age mendekati distribusi normal dengan puncak sekitar 30 sampai 40 tahun, sedangkan day relatif merata sepanjang bulan.



Perulangan menggambar satu boxplot untuk setiap variabel numerik dengan sns.boxplot(y=df[col]). Kotak menunjukkan rentang kuartil pertama sampai ketiga, garis tengah adalah median, dan titik di luar *whisker* adalah calon outlier. boxplot memperlihatkan banyak titik di atas batas atas *whisker* pada balance, duration, campaign, dan previous, yang menandakan outlier di sisi nilai besar. Variabel age hanya memiliki sedikit titik di atas *whisker*, dan day tidak memiliki outlier.

```
[22]: # NIM: 202431185 | Muh. Razzan Rezqy Pratama

baris = []
for col in kolom_numerik:
    Q1, Q3 = df[col].quantile(0.25), df[col].quantile(0.75)
    IQR = Q3 - Q1
    batas_bawah, batas_atas = Q1 - 1.5 * IQR, Q3 + 1.5 * IQR
    jumlah = ((df[col] < batas_bawah) | (df[col] > batas_atas)).sum()
    baris.append([col, Q1, Q3, batas_bawah, batas_atas, jumlah, round(jumlah / len(df) * 100, 2)])

tabel_outlier = pd.DataFrame(baris, columns=["kolom", "Q1", "Q3", "batas_bawah", "batas_atas", "jumlah_outlier", "persen"])
tabel_outlier
```

[22]:

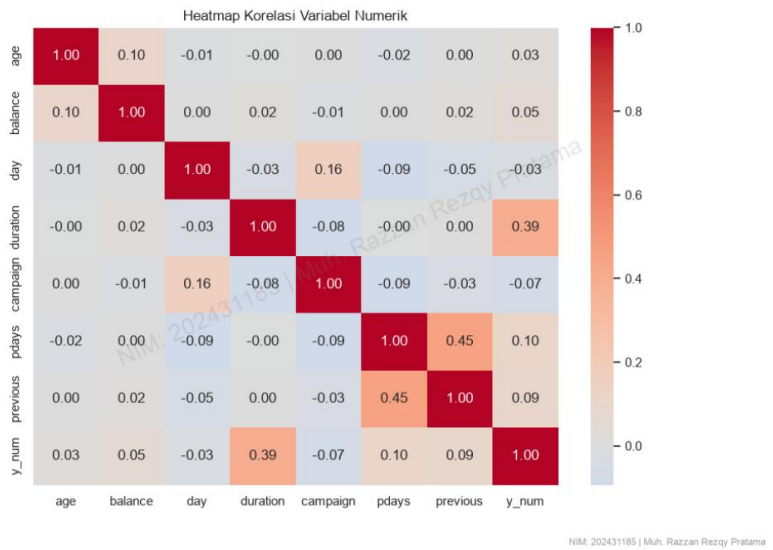
| | kolom | Q1 | Q3 | batas_bawah | batas_atas | jumlah_outlier | persen |
|---|---|---|---|---|---|---|---|
| 0 | age | 33.0 | 48.0 | 10.5 | 70.5 | 487 | 1.08 |
| 1 | balance | 72.0 | 1428.0 | -1962.0 | 3462.0 | 4729 | 10.46 |
| 2 | day | 8.0 | 21.0 | -11.5 | 40.5 | 0 | 0.00 |
| 3 | duration | 103.0 | 319.0 | -221.0 | 643.0 | 3235 | 7.16 |
| 4 | campaign | 1.0 | 3.0 | -2.0 | 6.0 | 3064 | 6.78 |
| 5 | pdays | -1.0 | -1.0 | -1.0 | -1.0 | 8257 | 18.26 |
| 6 | previous | 0.0 | 0.0 | 0.0 | 0.0 | 8257 | 18.26 |

Untuk setiap variabel numerik dihitung kuartil pertama (Q1), kuartil ketiga (Q3), dan IQR = Q3 - Q1. Batas bawah adalah Q1 - 1,5 x IQR dan batas atas adalah Q3 + 1,5 x IQR; nilai di luar kedua batas dihitung sebagai outlier. Hasilnya dirangkum dalam tabel tabel_outlier berisi Q1, Q3, batas, jumlah outlier, dan persentasenya. balance memiliki 4.729 outlier (10,46%), duration 3.235 (7,16%), campaign 3.064 (6,78%), dan age 487 (1,08%). Pada pdays dan previous, 8.257 baris (18,26%) terdeteksi sebagai outlier karena sebagian besar nasabah bernilai -1 dan 0 sehingga IQR-nya nol. Outlier ini tetap dipertahankan karena merupakan nilai nyata, bukan kesalahan input.

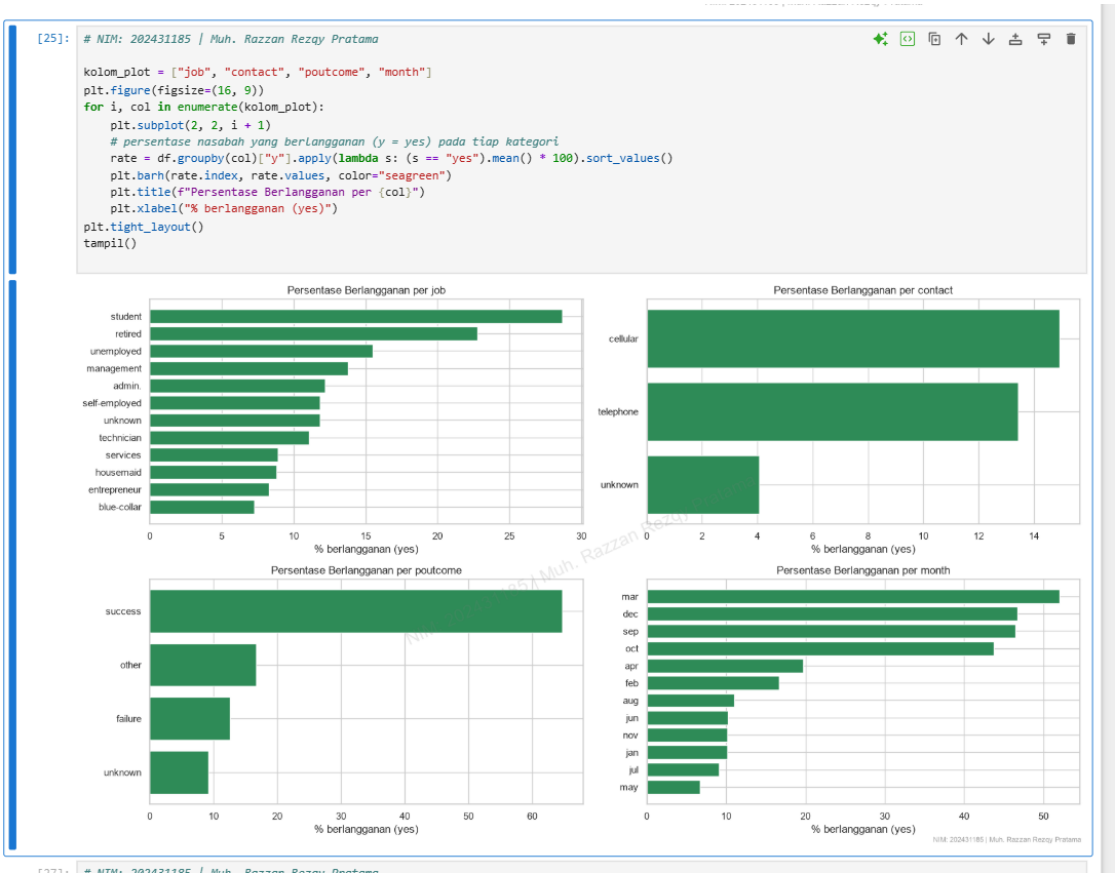
```
[66]: # NIM: 202431185 | Muh. Razzan Rezqy Pratama

# y diubah sementara menjadi angka 0/1 agar korelasinya dengan variabel numerik bisa dilihat
df_korelasi = df[kolom_numerik].assign(y_num=(df["y"] == "yes").astype(int))

plt.figure(figsize=(10, 7))
sns.heatmap(df_korelasi.corr(), annot=True, fmt=".2f", cmap="coolwarm", center=0)
plt.title("Heatmap Korelasi Variabel Numerik")
tampil()
```



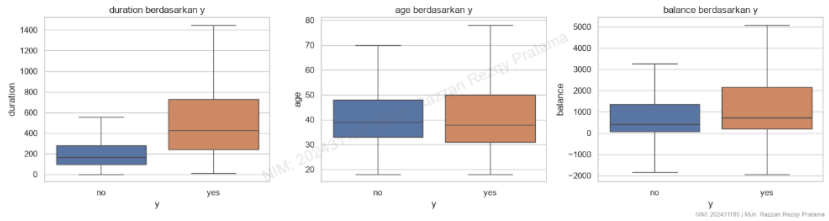
Kolom y sementara diubah menjadi angka 0/1 dengan assign(y_num=...) agar korelasinya dengan variabel numerik dapat dihitung. df_korelasi.corr() menghasilkan matriks korelasi Pearson, dan sns.heatmap menampilkannya sebagai peta warna dengan angka di dalam kotak (annot=True, fmt=".2f"). Skala warna coolwarm dipusatkan di nol (center=0) sehingga korelasi positif berwarna merah dan negatif berwarna biru. Korelasi antar-fitur numerik umumnya lemah (|r| < 0,2), kecuali pdays dan previous (r = 0,45). Fitur yang paling berkorelasi dengan target adalah duration (r = 0,39), sedangkan balance hanya berkorelasi lemah dengan age (r = 0,10).




Untuk empat kolom kategorikal (job, contact, poutcome, month), groupby(col)["y"].apply(lambda s: (s == "yes").mean() * 100) menghitung persentase nasabah yang berlangganan pada tiap kategori, kemudian diurutkan dengan sort_values() dan digambar sebagai diagram batang horizontal (plt.barh). Persentase berlangganan tertinggi terdapat pada nasabah *student* (sekitar 29%) dan *retired* (sekitar 23%), pada poutcome = success (sekitar 65%), serta pada bulan Maret, Desember, September, dan Oktober (di atas 40%). Sebaliknya, bulan Mei yang paling banyak dihubungi memiliki tingkat berlangganan hanya sekitar 7%.

```
[67]: # NIM: 202431185 | Muh. Razzan Rezqy Pratama

plt.figure(figsize=(15, 4))
for i, col in enumerate(["duration", "age", "balance"]):
    plt.subplot(1, 3, i + 1)
    sns.boxplot(x="y", y=col, hue="y", data=df, showfliers=False, legend=False)
    plt.title(f"{col} berdasarkan y")
plt.tight_layout()
tampil()
```




Boxplot dibuat dengan x="y" sehingga sebaran duration, age, dan balance dapat dibandingkan antara kelas no dan yes. Parameter showfliers=False menyembunyikan titik outlier supaya kotak tidak tertekan oleh nilai ekstrem. Nasabah yang berlangganan (yes) memiliki durasi telepon yang jauh lebih panjang daripada yang tidak. Perbedaan age dan balance antar-kelas tidak sebesar duration.

## 3.4 Preprocessing Data

```
[31]: # NIM: 202431185 | Muh. Razzan Rezqy Pratama

df_model = df.drop_duplicates().copy()                  # 1. buang duplikat (hasilnya tidak ada yang terbuang)
df_model["y"] = df_model["y"].map({"no": 0, "yes": 1})  # 4. target menjadi angka
print("Ukuran data setelah dibersihkan:", df_model.shape)
print(df_model["y"].value_counts())
df_model.head()
```

```
Ukuran data setelah dibersihkan: (45211, 17)
y
0    39922
1     5289
Name: count, dtype: int64
```

[31]:

| | age | job | marital | education | default | balance | housing | loan | contact | day | month | duration | campaign | pdays | previous | poutcome | y |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 58 | management | married | tertiary | no | 2143 | yes | no | unknown | 5 | may | 261 | 1 | -1 | 0 | unknown | 0 |
| 1 | 44 | technician | single | secondary | no | 29 | yes | no | unknown | 5 | may | 151 | 1 | -1 | 0 | unknown | 0 |
| 2 | 33 | entrepreneur | married | secondary | no | 2 | yes | yes | unknown | 5 | may | 76 | 1 | -1 | 0 | unknown | 0 |
| 3 | 47 | blue-collar | married | unknown | no | 1506 | yes | no | unknown | 5 | may | 92 | 1 | -1 | 0 | unknown | 0 |
| 4 | 33 | unknown | single | unknown | no | 1 | no | no | unknown | 5 | may | 198 | 1 | -1 | 0 | unknown | 0 |

df.drop_duplicates() membuang baris duplikat (hasilnya tidak ada baris yang terbuang) dan .copy() membuat salinan agar data asli df tidak berubah. map({"no": 0, "yes": 1}) mengubah target teks menjadi angka sehingga dapat diproses algoritma dan metrik evaluasi. Berdasarkan hasil outputnya menunjukkan bahwa tidak ada baris yang terbuang (ukuran tetap 45.211 x 17). Target sudah berbentuk angka: 0 = tidak berlangganan (39.922) dan 1 = berlangganan (5.289).

```
[33]: # NIM: 202431185 | Muh. Razzan Rezqy Pratama
# Daftar kolom kategorikal & numerik untuk model klasifikasi (Decision Tree)
fitur_kategori = ["job", "marital", "education", "default", "housing",
                  "loan", "contact", "month", "poutcome"]
fitur_numerik  = ["age", "balance", "day", "duration", "campaign", "pdays", "previous"]

X = df_model.drop(columns="y")       # fitur
y = df_model["y"]                    # target

# stratify=y: proporsi yes/no di data latih & uji dibuat sama dengan data asli
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=RANDOM_STATE, stratify=y)

print("X_train:", X_train.shape, "| X_test:", X_test.shape)
print("Proporsi yes (data latih):", round(y_train.mean() * 100, 2), "%")
print("Proporsi yes (data uji)  :", round(y_test.mean() * 100, 2), "%")
```

```
X_train: (36168, 16) | X_test: (9043, 16)
Proporsi yes (data latih): 11.7 %
Proporsi yes (data uji)  : 11.7 %
```

Kolom dikelompokkan menjadi fitur_kategori (9 kolom teks) dan fitur_numerik (7 kolom angka). X berisi seluruh fitur (kolom y dibuang dengan drop) dan y berisi target. train_test_split membagi data menjadi 80% data latih dan 20% data uji (test_size=0.2).

Parameter stratify=y menjaga proporsi kelas yes dan no di data latih dan data uji sama dengan data asli, hal yang penting karena kelas tidak seimbang. Data terbagi menjadi 36.168 data latih dan 9.043 data uji. Berkat stratify=y, proporsi kelas yes di kedua bagian sama persis, yaitu 11,7%.

[68]:
```
# NIM: 202431185 | Muh. Razzan Rezqy Pratama

preprocessor_dt = ColumnTransformer([
    ("kategori", OneHotEncoder(handle_unknown="ignore"), fitur_kategori),
    ("numerik", "passthrough", fitur_numerik)
])

preprocessor_dt.fit(X_train)
print("Jumlah fitur sebelum encoding :", X_train.shape[1])
print("Jumlah fitur setelah encoding :", len(preprocessor_dt.get_feature_names_out()))
```

```
Jumlah fitur sebelum encoding : 16
Jumlah fitur setelah encoding : 51
```

ColumnTransformer menerapkan perlakuan berbeda pada tiap kelompok kolom: kolom kategorikal diubah dengan OneHotEncoder menjadi kolom biner 0/1, sedangkan kolom numerik diteruskan apa adanya ("passthrough") karena Decision Tree tidak membutuhkan penskalaan. Opsi handle_unknown="ignore" mencegah error bila pada data uji muncul kategori yang tidak ada di data latih. preprocessor_dt.fit(X_train) mempelajari kategori hanya dari data latih untuk menghindari *data leakage*. Setelah One-Hot Encoding, jumlah fitur bertambah dari 16 menjadi 51 kolom karena setiap kategori diubah menjadi kolom 0/1 tersendiri.

## 3.5 Decision Tree (Klasifikasi)

[36]:
```
# NIM: 202431185 | Muh. Razzan Rezqy Pratama

pipe_dt_awal = Pipeline([
    ("prep", preprocessor_dt),
    ("model", DecisionTreeClassifier(random_state=RANDOM_STATE))   # tanpa max_depth
])
pipe_dt_awal.fit(X_train, y_train)

acc_train_awal = accuracy_score(y_train, pipe_dt_awal.predict(X_train))
acc_test_awal  = accuracy_score(y_test,  pipe_dt_awal.predict(X_test))
print(f"Kedalaman pohon  : {pipe_dt_awal['model'].get_depth()}")
print(f"Jumlah daun      : {pipe_dt_awal['model'].get_n_leaves()}")
print(f"Akurasi data latih: {acc_train_awal * 100:.2f}%")
print(f"Akurasi data uji  : {acc_test_awal * 100:.2f}%")
```

```
Kedalaman pohon  : 34
Jumlah daun      : 2876
Akurasi data latih: 100.00%
Akurasi data uji  : 87.37%
```

Pipeline menggabungkan preprocessor dan model menjadi satu rangkaian, sehingga proses encoding dan pelatihan selalu berjalan berurutan dan konsisten. DecisionTreeClassifier(random_state=RANDOM_STATE) dibuat tanpa max_depth sehingga pohon tumbuh sampai semua daun murni. Setelah fit, akurasi dihitung pada data latih dan data uji dengan accuracy_score, dan get_depth() serta get_n_leaves() menunjukkan ukuran pohon. Perbandingan akurasi latih dan uji dipakai untuk memeriksa overfitting. Pohon tanpa batasan tumbuh sampai kedalaman 34 dengan 2.876 daun. Akurasi data latih mencapai 100% tetapi akurasi data uji hanya 87,37% (lebih rendah daripada tebakan kelas mayoritas 88,30%). Ini tanda overfitting: pohon menghafal data latih. Karena itu perlu dilakukan pembatasan kompleksitas pohon.

[38]:
```
# NIM: 202431185 | Muh. Razzan Rezqy Pratama

param_grid = {
    "model__criterion": ["gini", "entropy"],
    "model__max_depth": [3, 5, 7, 9],
    "model__min_samples_leaf": [1, 10, 50],
}
grid = GridSearchCV(
    Pipeline([("prep", preprocessor_dt),
              ("model", DecisionTreeClassifier(random_state=RANDOM_STATE))]),
    param_grid, cv=5, scoring="f1", n_jobs=-1)
grid.fit(X_train, y_train)

print("Parameter terbaik :", grid.best_params_)
print(f"F1-score CV terbaik: {grid.best_score_:.4f}")
model_dt = grid.best_estimator_       # model terbaik hasil tuning
```

```
Parameter terbaik : {'model__criterion': 'gini', 'model__max_depth': 7, 'model__min_samples_leaf': 1}
F1-score CV terbaik: 0.4840
```

Param_grid berisi kandidat criterion (gini, entropy), max_depth (3, 5, 7, 9), dan min_samples_leaf (1, 10, 50). Awalan model__ menunjuk parameter milik langkah model di dalam Pipeline. GridSearchCV mencoba seluruh 24 kombinasi dengan validasi silang 5 lipatan (cv=5), sehingga terdapat 120 kali pelatihan. scoring="f1" dipilih karena kelas yes jauh lebih sedikit sehingga akurasi saja kurang adil, dan n_jobs=-1 memakai seluruh inti prosesor. Model terbaik disimpan pada model_dt = grid.best_estimator. Dengan begitu, Kombinasi terbaik adalah criterion = gini, max_depth = 7, dan min_samples_leaf = 1 dengan F1-score validasi silang 0,4840.

[69]:
```
# NIM: 202431185 | Muh. Razzan Rezqy Pratama

y_pred_dt  = model_dt.predict(X_test)
y_proba_dt = model_dt.predict_proba(X_test)[:, 1]      # probabilitas kelas yes

acc_dt = accuracy_score(y_test, y_pred_dt)
print(f"Akurasi data latih : {accuracy_score(y_train, model_dt.predict(X_train)) * 100:.2f}%")
print(f"Akurasi data uji   : {acc_dt * 100:.2f}%")
print(f"Precision (yes)    : {precision_score(y_test, y_pred_dt):.4f}")
print(f"Recall (yes)       : {recall_score(y_test, y_pred_dt):.4f}")
print(f"F1-score (yes)     : {f1_score(y_test, y_pred_dt):.4f}")
print(f"ROC-AUC            : {roc_auc_score(y_test, y_proba_dt):.4f}")
print()
print(classification_report(y_test, y_pred_dt, target_names=["no", "yes"]))
```

```
Akurasi data latih : 91.15%
Akurasi data uji   : 89.86%
Precision (yes)    : 0.6083
Recall (yes)       : 0.3743
F1-score (yes)     : 0.4634
ROC-AUC            : 0.8629

              precision    recall  f1-score   support

          no       0.92      0.97      0.94      7985
         yes       0.61      0.37      0.46      1058

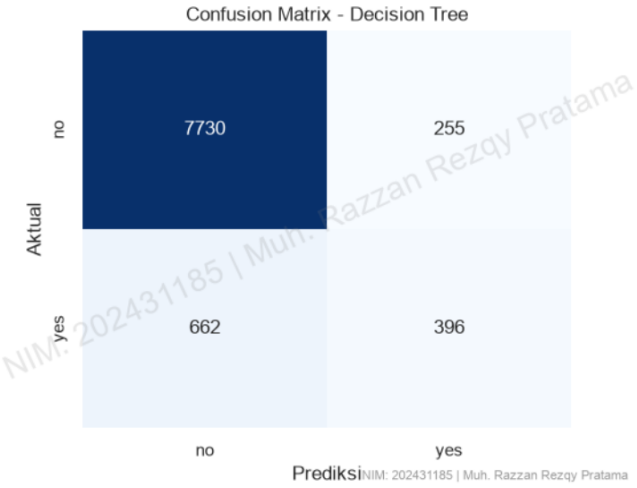
    accuracy                           0.90      9043
   macro avg       0.76      0.67      0.70      9043
weighted avg       0.88      0.90      0.89      9043
```

Berdasarkan predict menghasilkan kelas prediksi, sedangkan predict_proba(...)[:, 1] menghasilkan probabilitas kelas yes yang dibutuhkan untuk ROC-AUC. Kode ini menghitung akurasi (latih dan uji), precision, recall, F1-score, dan ROC-AUC, kemudian classification_report menampilkan ringkasan precision, recall, dan F1 untuk kedua kelas. Akurasi data uji 89,86% dan akurasi data latih 91,15%. Selisih keduanya kecil (sekitar 1,3 poin), jadi overfitting sudah teratasi. Untuk kelas yes, precision 0,61 (dari semua nasabah yang diprediksi berlangganan, 61% benar) dan recall 0,37 (hanya 37% dari nasabah yang benar-benar berlangganan berhasil terdeteksi). ROC-AUC 0,863 menunjukkan kemampuan membedakan kedua kelas yang baik. Peningkatan akurasi dibanding tebakan kelas mayoritas (88,30%) hanya sekitar 1,6 poin, jadi nilai model terutama terlihat pada ROC-AUC dan kemampuan menemukan nasabah yes.

[41]:
```
# NIM: 202431185 | Muh. Razzan Rezqy Pratama

cm = confusion_matrix(y_test, y_pred_dt)
print(cm)
plt.figure(figsize=(5.5, 4.5))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", cbar=False,
            xticklabels=["no", "yes"], yticklabels=["no", "yes"])
plt.xlabel("Prediksi")
plt.ylabel("Aktual")
plt.title("Confusion Matrix - Decision Tree")
tampil()
```

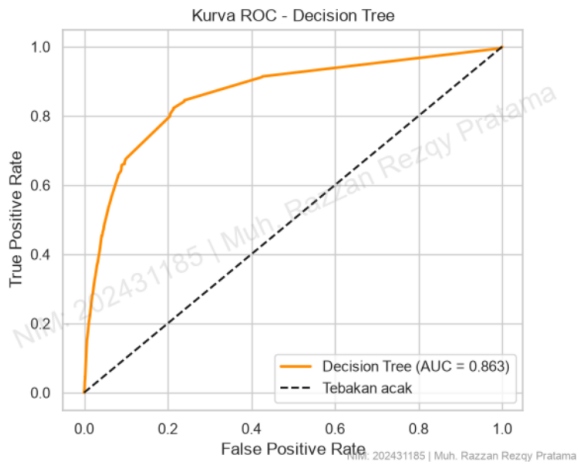
```
[[7730  255]
 [ 662  396]]
```




Confusion_matrix(y_test, y_pred_dt) menghitung jumlah prediksi benar dan salah untuk tiap kelas. Hasilnya digambar sebagai heatmap dengan sns.heatmap(..., annot=True, fmt="d") sehingga angka tampil sebagai bilangan bulat. Sumbu vertikal adalah kelas aktual dan sumbu horizontal adalah kelas prediksi. Dari 9.043 data uji: 7.730 *True Negative*, 396 *True Positive*, 255 *False Positive*, dan 662 *False Negative*. Kesalahan terbesar adalah *False Negative*, yaitu nasabah yang sebenarnya mau berlangganan tetapi diprediksi tidak.

[42]:
```
# NIM: 202431185 | Muh. Razzan Rezqy Pratama

fpr, tpr, _ = roc_curve(y_test, y_proba_dt)
plt.figure(figsize=(6, 5))
plt.plot(fpr, tpr, color="darkorange", lw=2, label=f"Decision Tree (AUC = {roc_auc_score(y_test, y_proba_dt):.3f})")
plt.plot([0, 1], [0, 1], "k--", label="Tebakan acak")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("Kurva ROC - Decision Tree")
plt.legend()
tampil()
```




roc_curve menghitung *False Positive Rate* dan *True Positive Rate* pada berbagai ambang probabilitas. Kurva digambar dengan plt.plot, sedangkan garis putus-putus diagonal

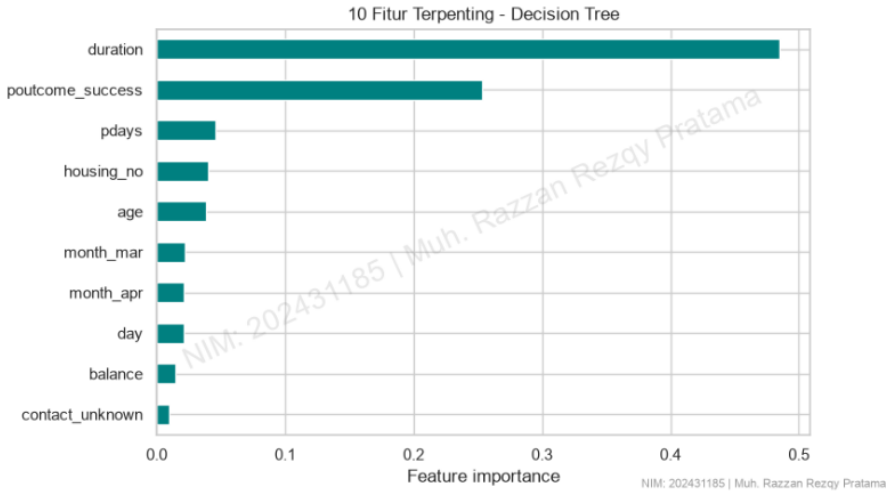
mewakili tebakan acak sebagai pembanding. Nilai AUC ditampilkan pada legenda. Kurva ROC berada jauh di atas garis diagonal (tebakan acak) dengan AUC = 0,863.

```
[43]: # NIM: 202431185 | Muh. Razzan Rezqy Pratama

nama_fitur = [n.split("__", 1)[1] for n in model_dt["prep"].get_feature_names_out()]   # buang awalan "kategori__"/"numerik__"
pentingnya = pd.Series(model_dt["model"].feature_importances_, index=nama_fitur).sort_values(ascending=False)
print(pentingnya.head(10).round(4))

plt.figure(figsize=(8, 5))
pentingnya.head(10).sort_values().plot(kind="barh", color="teal")
plt.title("10 Fitur Terpenting - Decision Tree")
plt.xlabel("Feature importance")
tampil()
```

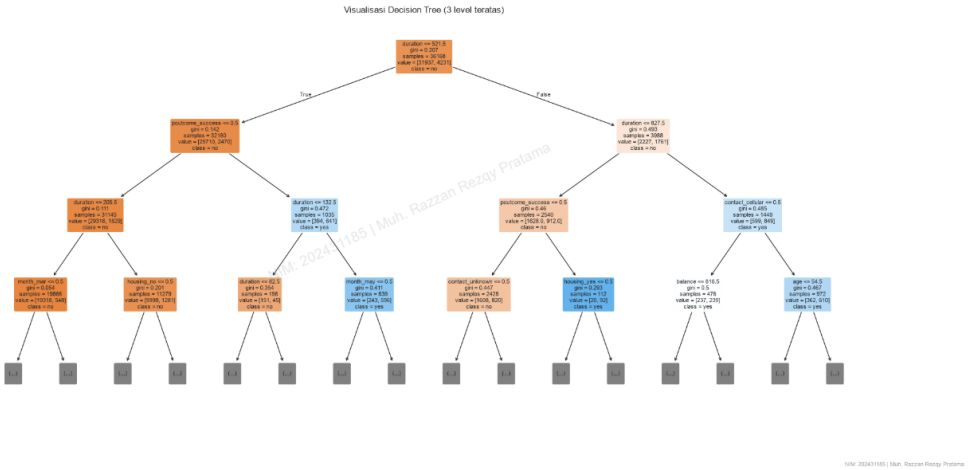
```
duration           0.4846
poutcome_success   0.2536
pdays              0.0462
housing_no         0.0400
age                0.0386
month_mar          0.0222
month_apr          0.0215
day                0.0212
balance            0.0149
contact_unknown    0.0099
dtype: float64
```




get_feature_names_out() mengambil nama kolom hasil One-Hot Encoding, lalu split("__", 1)[1] membuang awalan kategori__ atau numerik__ agar nama lebih singkat. Atribut feature_importances_ milik pohon menunjukkan kontribusi tiap fitur dalam menurunkan impuritas. Hasilnya disusun dalam pd.Series, diurutkan, dan sepuluh teratas digambar sebagai diagram batang. Fitur paling berpengaruh adalah duration (0,485) dan poutcome_success (0,254), diikuti pdays (0,046), housing_no (0,040), dan age (0,039). Dua fitur teratas saja menyumbang sekitar 74% total kepentingan.

```
[44]: # NIM: 202431185 | Muh. Razzan Rezqy Pratama

plt.figure(figsize=(22, 10))
plot_tree(model_dt["model"], max_depth=3,            # hanya 3 level teratas supaya terbaca
          feature_names=nama_fitur, class_names=["no", "yes"],
          filled=True, rounded=True, fontsize=8)
plt.title("Visualisasi Decision Tree (3 level teratas)")
tampil()
```




plot_tree menggambar pohon keputusan. Argumen max_depth=3 membatasi tampilan pada tiga level teratas supaya terbaca, feature_names dan class_names memberi label yang mudah dipahami, dan filled=True memberi warna sesuai kelas mayoritas pada tiap simpul. Setiap simpul menampilkan aturan pemisah, nilai gini, jumlah sampel, dan jumlah sampel per kelas (value). Akar pohon memisahkan data berdasarkan duration <= 521,5 detik. Cabang kiri (durasi pendek) didominasi kelas no, sedangkan cabang kanan dengan durasi panjang memiliki proporsi yes yang jauh lebih besar. Hasil keberhasilan kampanye sebelumnya (poutcome_success) muncul pada level berikutnya.

```
[46]: # NIM: 202431185 | Muh. Razzan Rezqy Pratama

def latih_dt(buang=None):
    """Melatih Decision Tree dengan parameter terbaik, dengan opsi membuang kolom tertentu."""
    buang = buang or []
    kat = [c for c in fitur_kategori if c not in buang]
    num = [c for c in fitur_numerik if c not in buang]
    pipe = Pipeline([
        ("prep", ColumnTransformer([("kategori", OneHotEncoder(handle_unknown="ignore"), kat),
                                    ("numerik", "passthrough", num)])),
        ("model", DecisionTreeClassifier(
            criterion=grid.best_params_["model__criterion"],
            max_depth=grid.best_params_["model__max_depth"],
            min_samples_leaf=grid.best_params_["model__min_samples_leaf"],
            random_state=RANDOM_STATE))
    ])
    pipe.fit(X_train, y_train)
    pred, proba = pipe.predict(X_test), pipe.predict_proba(X_test)[:, 1]
    return [accuracy_score(y_test, pred), precision_score(y_test, pred),
            recall_score(y_test, pred), f1_score(y_test, pred), roc_auc_score(y_test, proba)]

perbandingan = pd.DataFrame(
    [latih_dt(), latih_dt(["duration"])],
    columns=["Accuracy", "Precision", "Recall", "F1-score", "ROC-AUC"],
    index=["Dengan duration", "Tanpa duration"]).round(4)
perbandingan
```

[46]:

| | Accuracy | Precision | Recall | F1-score | ROC-AUC |
|---|---|---|---|---|---|
| **Dengan duration** | 0.8986 | 0.6083 | 0.3743 | 0.4634 | 0.8629 |
| **Tanpa duration** | 0.8930 | 0.6148 | 0.2278 | 0.3324 | 0.7120 |

Fungsi latih_dt(buang) melatih ulang Decision Tree dengan parameter terbaik hasil GridSearchCV, dengan opsi membuang kolom tertentu dari fitur. Fungsi ini mengembalikan lima metrik (accuracy, precision, recall, F1, ROC-AUC) pada data uji. Fungsi dipanggil dua kali: dengan seluruh fitur dan tanpa duration, lalu hasilnya disusun dalam satu tabel perbandingan. Eksperimen ini dilakukan karena duration baru diketahui setelah panggilan telepon selesai. Dengan begitu, tanpa duration, akurasi hanya turun sedikit (89,86% menjadi 89,30%), tetapi recall turun dari 0,374 menjadi 0,228, F1 turun dari 0,463 menjadi 0,332, dan ROC-AUC turun dari 0,863 menjadi 0,712. Jadi sebagian besar

kemampuan model yang tinggi berasal dari duration, yang tidak tersedia sebelum nasabah dihubungi. Model tanpa duration lebih realistis untuk dipakai sebagai alat prediksi di lapangan.

[47]:
```
# NIM: 202431185 | Muh. Razzan Rezqy Pratama

nasabah_baru = pd.DataFrame([{
    "age": 35, "job": "management", "marital": "married", "education": "tertiary",
    "default": "no", "balance": 2500, "housing": "no", "loan": "no",
    "contact": "cellular", "day": 15, "month": "may", "duration": 400,
    "campaign": 2, "pdays": -1, "previous": 0, "poutcome": "unknown"}])

hasil = model_dt.predict(nasabah_baru)[0]
peluang = model_dt.predict_proba(nasabah_baru)[0, 1]
print("Prediksi :", "BERLANGGANAN (yes)" if hasil == 1 else "TIDAK berlangganan (no)")
print(f"Peluang berlangganan: {peluang * 100:.2f}%")
```

```
Prediksi : TIDAK berlangganan (no)
Peluang berlangganan: 12.71%
```

Satu nasabah contoh dibuat sebagai DataFrame satu baris dengan seluruh 16 fitur. Pipeline otomatis melakukan encoding sebelum prediksi, sehingga data baru tidak perlu diubah secara manual. predict memberikan kelas prediksi dan predict_proba(...)[0, 1] memberikan peluang nasabah berlangganan. untuk nasabah contoh, model memprediksi tidak berlangganan dengan peluang berlangganan 12,71%.

## 3.6 Regresi Linear

[70]:
```
# NIM: 202431185 | Muh. Razzan Rezqy Pratama

print("Korelasi variabel numerik terhadap balance:")
print(df[kolom_numerik].corr()["balance"].drop("balance").sort_values().round(4))

fitur_numerik_reg = ["age", "day", "duration", "campaign", "pdays", "previous"]
X_reg = df_model.drop(columns=["balance", "y"])
y_reg = df_model["balance"]

X_train_r, X_test_r, y_train_r, y_test_r = train_test_split(
    X_reg, y_reg, test_size=0.2, random_state=RANDOM_STATE)
print("\nX_train_r:", X_train_r.shape, "| X_test_r:", X_test_r.shape)
```

```
Korelasi variabel numerik terhadap balance:
campaign   -0.0146
pdays       0.0034
day         0.0045
previous    0.0167
duration    0.0216
age         0.0978
Name: balance, dtype: float64

X_train_r: (36168, 15) | X_test_r: (9043, 15)
```

Korelasi setiap variabel numerik terhadap balance dihitung untuk melihat potensi hubungan linear. Pada regresi, balance menjadi target (y_reg), sehingga kolom balance dan y dibuang dari fitur (X_reg). Data dibagi 80:20 dengan random_state=42 tanpa stratify karena target berupa angka kontinu. Semua fitur numerik hanya berkorelasi sangat lemah dengan balance. Korelasi tertinggi adalah age (r = 0,098), sedangkan lainnya di bawah 0,03. Ini memberi petunjuk awal bahwa Regresi Linear tidak akan menghasilkan R-squared yang tinggi.

[73]:
```
# NIM: 202431185 | Muh. Razzan Rezqy Pratama

preprocessor_lr = ColumnTransformer([
    ("kategori", OneHotEncoder(drop="first", handle_unknown="ignore"), fitur_kategori),
    ("numerik", StandardScaler(), fitur_numerik_reg)
])

model_lr = Pipeline([
    ("prep", preprocessor_lr),
    ("model", LinearRegression())
])
model_lr.fit(X_train_r, y_train_r)       # fase training
print("Model Regresi Linear berhasil dilatih.")
print("Intercept:", round(model_lr["model"].intercept_, 4))
```

```
Model Regresi Linear berhasil dilatih.
Intercept: 1192.8248
```

Pada regresi, kolom kategorikal diubah dengan OneHotEncoder(drop="first"), yang membuang satu kategori sebagai acuan untuk menghindari kolinearitas antar kolom dummy. Kolom numerik distandarkan dengan StandardScaler (rata-rata 0, simpangan baku 1) sehingga koefisien antar fitur lebih sebanding. LinearRegression kemudian dilatih lewat fit, dan intercept_ menunjukkan konstanta (beta nol) dalam persamaan regresi. Model Regresi Linear selesai dilatih dengan intercept 1.192,82. Artinya, untuk nasabah dengan semua fitur numerik pada nilai rata-ratanya dan seluruh kategori pada kategori acuan, prediksi saldonya sekitar 1.193 euro.

[75]:
```
# NIM: 202431185 | Muh. Razzan Rezqy Pratama

def evaluasi(y_asli, y_prediksi):
    mse = mean_squared_error(y_asli, y_prediksi)
    return [mean_absolute_error(y_asli, y_prediksi), mse, np.sqrt(mse), r2_score(y_asli, y_prediksi)]

y_pred_train_r = model_lr.predict(X_train_r)
y_pred_r       = model_lr.predict(X_test_r)

hasil_reg = pd.DataFrame(
    [evaluasi(y_train_r, y_pred_train_r), evaluasi(y_test_r, y_pred_r)],
    columns=["MAE", "MSE", "RMSE", "R2"],
    index=["Data latih", "Data uji"]).round(4)
hasil_reg
```

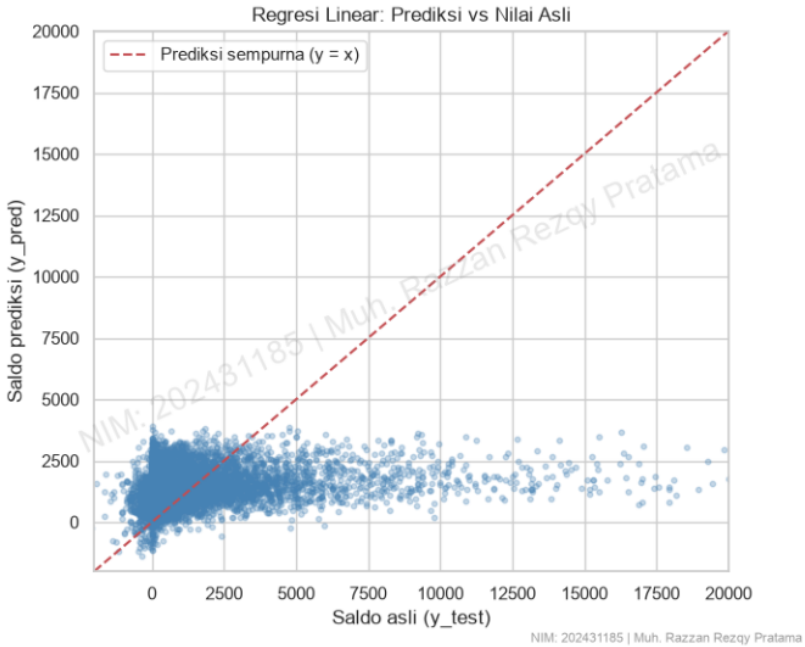
[75]:

| | MAE | MSE | RMSE | R2 |
|---|---|---|---|---|
| **Data latih** | 1475.6531 | 8.673723e+06 | 2945.1186 | 0.0509 |
| **Data uji** | 1486.8634 | 9.338625e+06 | 3055.9165 | 0.0468 |

Fungsi evaluasi menghitung empat ukuran: MAE (rata-rata selisih absolut), MSE (rata-rata kuadrat selisih), RMSE (akar dari MSE, satuannya sama dengan balance), dan R-squared (proporsi variasi target yang dijelaskan model). Evaluasi dilakukan pada data latih dan data uji agar dapat membandingkan keduanya. Pada data uji, MAE = 1.486,86 euro, RMSE = 3.055,92 euro, dan R-squared = 0,0468. Artinya model hanya menjelaskan sekitar 4,7% variasi saldo nasabah. Nilai R-squared data latih (0,0509) hampir sama dengan data uji, jadi model tidak overfitting tetapi *underfitting*: hubungan linear antara fitur yang tersedia dan saldo memang lemah.

[52]:
```
# NIM: 202431185 | Muh. Razzan Rezqy Pratama
cetak_wm()
plt.figure(figsize=(7, 6))
plt.scatter(y_test_r, y_pred_r, alpha=0.3, s=10, color="steelblue")
plt.plot([-2000, 20000], [-2000, 20000], "r--", label="Prediksi sempurna (y = x)")
plt.xlim(-2000, 20000)
plt.ylim(-2000, 20000)         # sumbu dibatasi agar sebaran mayoritas data terlihat
plt.xlabel("Saldo asli (y_test)")
plt.ylabel("Saldo prediksi (y_pred)")
plt.title("Regresi Linear: Prediksi vs Nilai Asli")
plt.legend()
tampil()
```

```
[ NIM: 202431185 | Muh. Razzan Rezqy Pratama ]
```



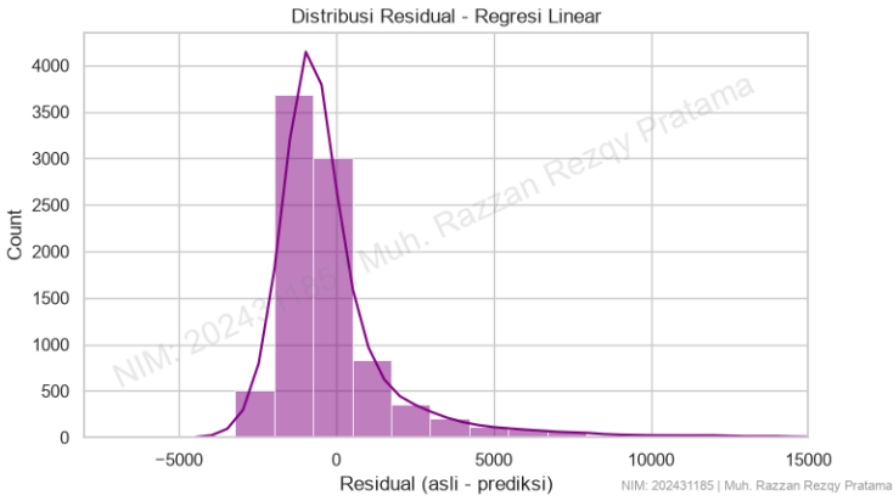

plt.scatter menggambar setiap nasabah uji sebagai satu titik dengan sumbu x berupa saldo asli dan sumbu y berupa saldo prediksi. Garis merah putus-putus adalah garis y = x yang menunjukkan prediksi sempurna. Sumbu dibatasi pada -2.000 sampai 20.000 supaya sebagian besar titik terlihat dengan jelas. Pada gambar diatas titik-titik tidak mengikuti garis diagonal. Prediksi terkumpul pada rentang sempit (sekitar 0 sampai 3.000 euro), sedangkan saldo asli tersebar sangat lebar. Model tidak mampu menangkap saldo yang sangat tinggi.

[53]:
```
# NIM: 202431185 | Muh. Razzan Rezqy Pratama

residual = y_test_r - y_pred_r        # selisih nilai asli dan prediksi
print(f"Rata-rata residual : {residual.mean():.2f}")
print(f"Median residual    : {residual.median():.2f}")

plt.figure(figsize=(8, 4.5))
sns.histplot(residual, bins=80, kde=True, color="purple")
plt.xlim(-8000, 15000)
plt.title("Distribusi Residual - Regresi Linear")
plt.xlabel("Residual (asli - prediksi)")
tampil()
```

```
Rata-rata residual : -7.56
Median residual    : -661.60
```



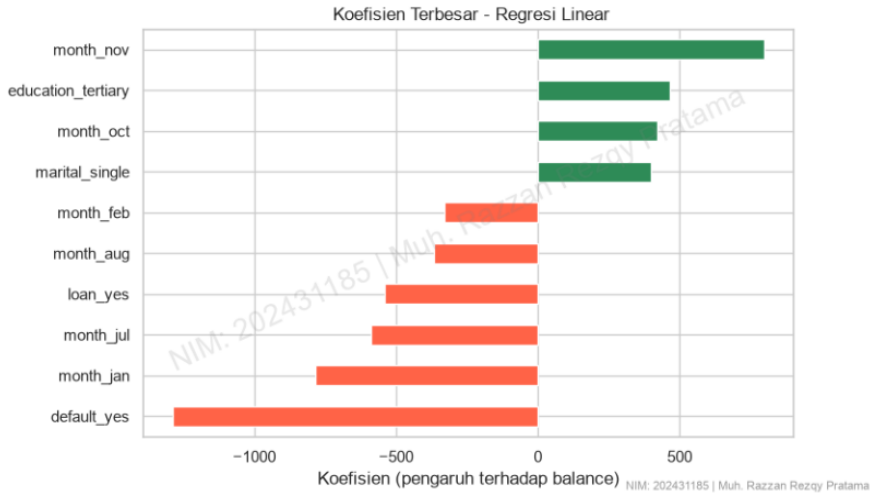

Residual dihitung sebagai selisih nilai asli dan nilai prediksi (y_test_r - y_pred_r). Rata-rata dan median residual dicetak, lalu sns.histplot menggambar distribusinya. Residual yang berpusat dekat nol dan simetris menandakan model tidak bias; ekor yang panjang menandakan adanya kesalahan besar pada sebagian data. residual berpusat dekat nol dengan rata-rata -7,56, tetapi median -661,60 dan ekor kanan yang panjang menunjukkan bahwa kesalahan terbesar muncul pada nasabah dengan saldo sangat tinggi yang diprediksi terlalu rendah.

[55]:
```
# NIM: 202431185 | Muh. Razzan Rezqy Pratama

nama_fitur_r = [n.split("__", 1)[1] for n in model_lr["prep"].get_feature_names_out()]
koefisien = pd.Series(model_lr["model"].coef_, index=nama_fitur_r)
top = koefisien.reindex(koefisien.abs().sort_values(ascending=False).index).head(10)
print(top.round(2))

plt.figure(figsize=(8, 5))
top.sort_values().plot(kind="barh", color=["tomato" if v < 0 else "seagreen" for v in top.sort_values()])
plt.title(" Koefisien Terbesar - Regresi Linear")
plt.xlabel("Koefisien (pengaruh terhadap balance)")
tampil()
```

```
default_yes          -1286.93
month_nov              798.11
month_jan             -783.95
month_jul             -589.79
loan_yes              -539.42
education_tertiary     464.17
month_oct              420.15
marital_single         399.12
month_aug             -365.99
month_feb             -329.47
dtype: float64
```




Atribut coef_ berisi koefisien tiap fitur, dipasangkan dengan nama kolomnya pada pd.Series. Koefisien diurutkan berdasarkan nilai mutlak dan sepuluh terbesar ditampilkan, dengan warna hijau untuk koefisien positif dan merah untuk koefisien negatif. Koefisien menyatakan perubahan prediksi balance ketika fitur tersebut naik satu satuan (atau berubah dari kategori acuan), dengan fitur lain tetap. Koefisien terbesar adalah default_yes (-1.286,93), yaitu nasabah yang pernah gagal bayar cenderung memiliki saldo lebih rendah sekitar 1.287 euro. Variabel loan_yes (-539,42) juga menurunkan saldo, sedangkan education_tertiary (+464,17) menaikkan saldo. Beberapa bulan kontak (misalnya November +798,11 dan Januari -783,95) ikut berpengaruh.

[76]:
```
# NIM: 202431185 | Muh. Razzan Rezqy Pratama

nasabah_reg = nasabah_baru.drop(columns="balance")
prediksi_saldo = model_lr.predict(nasabah_reg)[0]
print(f"Prediksi saldo nasabah: {prediksi_saldo:,.2f} euro")
```

```
Prediksi saldo nasabah: 1,760.23 euro
```

Data nasabah contoh yang sama dengan bagian Decision Tree dipakai kembali, tetapi kolom balance dibuang karena kolom itu adalah target yang diprediksi. model_lr.predict menghasilkan perkiraan saldo nasabah dalam euro. Untuk nasabah contoh yang sama, model memprediksi saldo sekitar 1.760,23 euro.

## 3.7 Rangkuman Hasil

[57]:
```
# NIM: 202431185 | Muh. Razzan Rezqy Pratama

rangkuman = pd.DataFrame({
    "Algoritma": ["Decision Tree (klasifikasi)", "Regresi Linear (regresi)"],
    "Target": ["y (berlangganan deposito)", "balance (saldo nasabah)"],
    "Metrik utama (data uji)": [f"Accuracy = {acc_dt * 100:.2f}%", f"R2 = {r2_score(y_test_r, y_pred_r):.4f}"],
    "Metrik pendukung": [f"F1 = {f1_score(y_test, y_pred_dt):.3f}; ROC-AUC = {roc_auc_score(y_test, y_proba_dt):.3f}",
                         f"MAE = {mean_absolute_error(y_test_r, y_pred_r):.1f}; RMSE = {np.sqrt(mean_squared_error(y_test_r, y_pred_r)):.1f}"]
})
rangkuman
```

[57]:

| | Algoritma | Target | Metrik utama (data uji) | Metrik pendukung |
|---|---|---|---|---|
| 0 | Decision Tree (klasifikasi) | y (berlangganan deposito) | Accuracy = 89.86% | F1 = 0.463; ROC-AUC = 0.863 |
| 1 | Regresi Linear (regresi) | balance (saldo nasabah) | R2 = 0.0468 | MAE = 1486.9; RMSE = 3055.9 |

Hasil kedua model dirangkum dalam satu tabel rangkuman yang berisi algoritma, target, metrik utama, dan metrik pendukung. Nilai pada tabel diambil langsung dari variabel hasil evaluasi sebelumnya melalui *f-string*, sehingga selalu konsisten dengan hasil perhitungan. Decision Tree (tuning) mencapai akurasi 89,86% dengan ROC-AUC 0,863, sedangkan Regresi Linear mencapai R-squared 0,0468 pada prediksi balance. Decision Tree cocok untuk data campuran numerik-kategorikal dan nonlinear seperti ini, sementara Regresi Linear kurang cocok untuk memprediksi saldo karena fitur yang tersedia hanya berhubungan lemah dengan saldo.

3.8 Pembahasan

Hasil EDA menunjukkan bahwa kelas target tidak seimbang: 88,30% nasabah tidak berlangganan. Konsekuensinya, model yang selalu menebak no sudah mencapai akurasi 88,30%. Decision Tree hasil tuning mencapai akurasi 89,86%, sehingga peningkatan akurasi terhadap tebakan kelas mayoritas hanya sekitar 1,6 poin persentase. Nilai model lebih tampak pada ROC-AUC sebesar 0,863 dan kemampuannya menemukan sebagian nasabah yes (recall 0,37 dengan precision 0,61). Hal ini sejalan dengan peringatan bahwa akurasi saja tidak cukup untuk menilai model pada data tidak seimbang (Hancock et al., 2023).

Model awal tanpa batasan mencapai akurasi latih 100% tetapi akurasi uji hanya 87,37%, bahkan lebih rendah daripada tebakan kelas mayoritas. Ini adalah contoh *overfitting* yang dijelaskan pada landasan teori (Halabaku & Bytyçi, 2024). Setelah max_depth dibatasi menjadi 7 melalui GridSearchCV, selisih akurasi latih (91,15%) dan uji (89,86%) menjadi kecil sehingga generalisasi membaik.

Fitur duration memiliki kepentingan tertinggi (0,485), diikuti poutcome_success (0,254). Karena duration baru diketahui setelah panggilan selesai, model yang memakainya tidak dapat dipakai untuk memilih nasabah sebelum dihubungi. Pada eksperimen tanpa duration, akurasi relatif stabil (89,30%), tetapi recall turun dari 0,374 menjadi 0,228 dan ROC-AUC turun dari 0,863 menjadi 0,712. Jadi, kemampuan model yang tampak tinggi sebagian besar berasal dari fitur tersebut, dan model tanpa duration lebih realistis untuk penerapan di lapangan.

Pada Regresi Linear, R-squared pada data uji hanya 0,0468 (data latih 0,0509), dengan MAE 1.486,86 dan RMSE 3.055,92 euro. Nilai latih dan uji yang hampir sama menunjukkan bahwa model tidak overfitting, melainkan *underfitting*: fitur yang tersedia hanya berkorelasi lemah dengan saldo (korelasi tertinggi age, r = 0,098) dan hubungan yang ada kemungkinan tidak linear, seperti diingatkan oleh Abu-Faraj et al. (2022). Distribusi balance yang sangat miring dengan 10,46% outlier juga membuat RMSE

(3.055,92) jauh lebih besar daripada MAE (1.486,86), yang menunjukkan adanya sejumlah kesalahan besar pada nasabah bersaldo tinggi. Dengan demikian Regresi Linear kurang cocok untuk memprediksi saldo dari atribut pada dataset ini, sedangkan Decision Tree lebih sesuai untuk data campuran numerik dan kategorikal.

# BAB IV
# PENUTUP

## 4.1 Kesimpulan

Berdasarkan hasil EDA, preprocessing, dan pemodelan yang telah dilakukan, dapat disimpulkan sebagai berikut.

1. Dataset Bank Marketing berisi 45.211 nasabah dan 17 atribut tanpa NaN dan tanpa duplikat, tetapi memiliki nilai unknown yang besar pada poutcome (81,75%) dan contact (28,80%), kelas target yang tidak seimbang (88,30% : 11,70%), dan outlier terutama pada balance, duration, dan campaign.
2. Preprocessing dengan One-Hot Encoding, pembagian data 80:20 secara stratifikasi, dan Pipeline berhasil menyiapkan data tanpa kebocoran data dari data uji.
3. Decision Tree awal mengalami overfitting (akurasi latih 100%, uji 87,37%). Setelah tuning dengan GridSearchCV (gini, max_depth = 7, min_samples_leaf = 1), akurasi data uji menjadi 89,86% dengan precision 0,61, recall 0,37, F1-score 0,46, dan ROC-AUC 0,863.
4. Fitur paling berpengaruh pada Decision Tree adalah duration dan poutcome_success. Tanpa duration, ROC-AUC turun dari 0,863 menjadi 0,712 dan recall turun dari 0,374 menjadi 0,228, sehingga model tanpa duration lebih realistis untuk prediksi sebelum nasabah dihubungi.
5. Regresi Linear untuk memprediksi balance menghasilkan R-squared 0,0468, MAE 1.486,86, dan RMSE 3.055,92 euro pada data uji. Model hanya menjelaskan sekitar 4,7% variasi saldo, sehingga kurang cocok untuk dataset ini karena fitur yang tersedia berhubungan lemah dengan saldo.

## 4.2 Saran

Saran untuk pengembangan selanjutnya adalah sebagai berikut.

1. Mencoba penanganan ketidakseimbangan kelas, misalnya parameter class_weight="balanced" atau teknik *resampling*, serta mengevaluasi dengan PR-AUC untuk meningkatkan recall kelas yes.
2. Mencoba algoritma berbasis ensemble seperti Random Forest atau Gradient Boosting yang menurut literatur dapat mengurangi overfitting pohon tunggal (Halabaku & Bytyçi, 2024).

3. Untuk memprediksi balance, mencoba transformasi target (misalnya logaritma setelah penyesuaian nilai negatif), penanganan outlier, atau model non-linear.

## DAFTAR PUSTAKA

Abu-Faraj, M., Al-Hyari, A., & Alqadi, Z. (2022). Experimental analysis of methods used to solve linear regression models. *Computers, Materials & Continua, 72*(3), 5699–5712. https://doi.org/10.32604/cmc.2022.027364

Alsulami, A., Khalifa, R., & Alghamdi, W. (2026). Improving performance and accuracy in decision trees: A literature survey on impurity functions. *International Journal of Advanced Computer Science and Applications (IJACSA), 17*(4), 985–1000. https://thesai.org/Downloads/Volume17No4/Paper_95-Improving_Performance_and_Accuracy_in_Decision_Trees.pdf

Halabaku, E., & Bytyçi, E. (2024). Overfitting in machine learning: A comparative analysis of decision trees and random forests. *Intelligent Automation & Soft Computing, 39*(6), 987–1006. https://doi.org/10.32604/iasc.2024.059429

Hancock, J. T., Khoshgoftaar, T. M., & Johnson, J. M. (2023). Evaluating classifier performance with highly imbalanced Big Data. *Journal of Big Data, 10*(1). https://doi.org/10.1186/s40537-023-00724-5

Hayder, I. M., Al Ali, G. A. N., & Younis, H. A. (2023). Predicting reaction based on customer's transaction using machine learning approaches. *International Journal of Electrical and Computer Engineering, 13*(1), 1086–1096. https://doi.org/10.11591/ijece.v13i1.pp1086-1096

Hodson, T. O. (2022). Root mean square error (RMSE) or mean absolute error (MAE): When to use them or not. *Geoscientific Model Development, 15*, 5481–5487. https://doi.org/10.5194/gmd-15-5481-2022

Hou, S., Cai, Z., Wu, J., Du, H., & Xie, P. (2022). Applying machine learning to the development of prediction models for bank deposit subscription. *International Journal of Business Analytics, 9*(1), 1–14. https://doi.org/10.4018/IJBAN.288514

Karunasingha, D. S. K. (2022). Root mean square error or mean absolute error? Use their ratio as well. *Information Sciences, 585*, 609–629. https://doi.org/10.1016/j.ins.2021.11.036

Pargent, F., Pfisterer, F., Thomas, J., & Bischl, B. (2022). Regularized target encoding outperforms traditional methods in supervised machine learning with high cardinality features. *Computational Statistics, 37*(5), 2671–2692. https://doi.org/10.1007/s00180-022-01207-6

Rosita, Y. D., & Moonlight, L. S. (2024). Comparative analysis of LSTM neural network and SVM for USD exchange rate prediction: A study on different training data scenarios. *Scientific Journal of Informatics, 11*(1), 195–206. https://doi.org/10.15294/sji.v11i1.49975